

김기원

BACKEND DEVELOPER

010-8390-5599 | kjjk3250@gmail.com

GitHub github.com/kimlony/hub | Notion [Easy Hub](#)



ERP·외부 시스템 연동과 주문수집 자동화에 강점을 가진 백엔드 개발자입니다. 동기 API 기반 수집 과정에서 겪은 실패 추적과 데이터 유실 문제를 Kafka, Outbox Pattern, Retry/DLQ, Recovery 구조로 확장해 해결하고, 대량 데이터 처리 성능과 운영 안정성을 함께 개선해 왔습니다.

핵심 역량

- Java·Spring Boot 기반 ERP 물류/영업 기능과 REST API 연동 개발
- 외부 쇼핑몰·물류 주문수집 및 Node.js/Puppeteer 기반 웹 자동화
- Kafka·Outbox Pattern 기반 비동기 Job 처리와 Retry/DLQ/Recovery 설계
- Oracle SQL 튜닝·페이징 조회·CSV 변환을 통한 대량 데이터 처리 개선
- requestId, Lease·Fencing Token, DB Lock 을 활용한 실패 추적·중복 방지·안전한 재처리
- Testcontainers·GitHub Actions·Docker·AWS EC2 기반 테스트 및 배포 자동화

기술 스택

Backend	Java · Spring Boot · REST API · MyBatis
Data / Messaging	Oracle · PostgreSQL · Apache Kafka · Redis
Automation / UI	Node.js · Puppeteer · JavaScript · TypeScript · AngularJS · React
DevOps / Test	Docker · Testcontainers · GitHub Actions · Git · SVN · AWS EC2

경력 및 학력 요약 총 경력 4 년 9 개월

회사 / 학교	기간	직무 / 전공
주식회사 비즈비	2023.07 - 재직 중	백엔드 개발자 / 사원
진우프린트 주식회사	2020.10 - 2022.05	제작관리 / 주임
광운대학교	2015.03 - 2020.02	경영학과 / 졸업

경력 상세

주식회사 비즈비 | 2023.07 - 재직 중 | 정규직 | 백엔드 개발자·사원

담당 업무 ERP 및 외부 시스템 연동, 쇼핑몰 주문수집, 물류·영업 기능 개발, 대량 조회 및 엑셀 다운로드 개선

핵심 성과 100 만 건+ 조회 안정화 · 50 만 건+ 다운로드 개선 · API 미제공 거래처 수집 자동화

1. 외부 시스템 연동 및 주문수집 기능 개발 | 2023.07 - 현재

- 외부 쇼핑몰·물류 시스템의 주문·입고·출고 데이터를 REST API 로 수신하고, 데이터 정합성 검증 후 ERP DB 에 저장
- API 를 제공하지 않는 거래처 물의 로그인·메뉴 이동·엑셀 다운로드를 Node.js/Puppeteer 로 자동화하고 ERP 형식으로 변환
- 사방넷 설치 프로그램이 필요한 주문수집·송장 송신을 위해 Windows 기반 AWS EC2 에 Node.js Worker 환경 구성 및 배포

사용 기술 Java, Oracle, MyBatis, AngularJS, REST API, Node.js, Puppeteer, AWS EC2

2. 대용량 조회 Timeout 개선 및 현황 프로그램 개발 | 2023.07 - 2026.06

- ERP 물류·영업 모듈의 대량 데이터 조회 Timeout 을 분석하고 Oracle SQL 튜닝 및 페이징 조회 구조로 개선
- 검색 조건과 사용자별 환경설정을 적용해 100 만 건 이상 데이터를 보유한 환경에서도 안정적으로 조회할 수 있도록 개선
- 주문 데이터 50 만 건 이상을 그리드에서 엑셀로 내려받을 때 발생하던 처리 지연을 csv 변환 방식으로 개선

사용 기술 Java, Oracle, SQL, MyBatis, AngularJS

3. 프랜차이즈 물류·영업 ERP 기능 개발 | 2023.07 - 2026.06

- 사용자 주문 등록 시 재고 관련 데이터를 조회해 주문 가능 여부를 검증하는 비즈니스 로직 구현
- 전표 처리 단계의 무조건 반올림으로 발생하던 금액 불일치를 해결하기 위해 회사별 소수점 처리 정책 기능 개발에 참여
- 주문·출고·입고 데이터를 분석해 재고 및 물류 현황 화면을 개발하고, 백엔드 API 부터 AngularJS 화면까지 전체 흐름 구현

진우프린트 주식회사 | 2020.10 - 2022.05 | 정규직 | 제작관리·주임

- 패키지 생산 공정, 물품 입출고, 다음 공정 진행 여부, 고객 대응 및 납기 일정 관리
- 수작업에 의존하던 반복 확인 업무를 경험하며 시스템화의 필요성을 체감했고, 이를 계기로 개발자로 전향

주요 프로젝트

Easy Hub — Kafka 기반 주문수집 자동화 플랫폼 | 개인 프로젝트 | 진행 중

REST API 기반 동기 주문수집에서 경험한 실패 추적, 재처리, 중복 처리, 완료 여부 확인의 어려움을 해결하기 위해 만든 프로젝트입니다. 주문수집 요청을 Job 으로 관리하고 Kafka Worker 기반 비동기 파이프라인으로 확장했습니다.

Hub API → hub_job / hub_job_outbox → Kafka → Worker → ERP·외부 시스템

설계 및 구현

- Spring Boot 기반 Hub API 와 Node.js Worker 구조 설계, 주문수집 요청을 hub_job 으로 관리하고 상태를 QUEUED·PROCESSING·SUCCESS·FAILED 로 추적
- DB 저장과 Kafka 발행 사이의 메시지 유실 가능성을 줄이기 위해 Outbox Pattern 적용: hub_job 과 hub_job_outbox 를 같은 트랜잭션으로 저장하고 Publisher 가 Kafka 로 발행
- Kafka 기반 비동기 Job 처리와 DB Lock·Kafka Key 를 활용한 동시성 제어로 동일 계정의 중복 수집 방지
- Worker 장애 후 중단 상태를 복구하는 Recovery Scanner 와 반복 실패 작업을 격리하는 Retry/DLQ 구조 구현
- requestId 기준으로 Kafka lag, Worker 처리량, Job 상태, DLQ 발생 여부를 확인할 수 있는 모니터링 화면 구현
- 로그인 계정의 회사 식별자를 기준으로 주문·Job·로그·ERP 전송 데이터의 접근 범위를 분리
- Testcontainers 기반 PostgreSQL/Kafka 통합 테스트와 GitHub Actions 기반 CI/CD 파이프라인 구성

ERP 연동 및 사용자 흐름

- 주문수집, 정규화, ERP 변환, 수집 전송, 외부 API 조회, 엑셀 다운로드까지 하나의 흐름으로 추적
- 자동 반영·수동 전송·외부 API 조회·엑셀 다운로드 이력을 동일 기준으로 확인할 수 있도록 상태 및 결과 기록 구조 설계
- 회사 업무에서 경험한 동기 주문수집 구조를 실패 추적·재처리·중복 방지·운영 모니터링까지 포함한 비동기 구조로 확장

성과 및 검증

100,000 건	3.1 배	1,214 초 → 388 초
주문수집 E2E 부하 테스트	4 worker 처리량 개선	동일 조건 처리시간 단축

테스트 확인 Kafka lag 0, DLQ 실패 메시지 없음, 주문수집·정규화·ERP 변환·전송·외부 API 조회·엑셀 다운로드 이력의 일관된 추적

기술 스택 Java, Spring Boot, PostgreSQL, Kafka, Node.js, TypeScript, Docker, Testcontainers, GitHub Actions, React

GitHub [Easy Hub](#) [GitHub 저장소](#)

Notion [Easy Hub 프로젝트 문서](#)

자기소개

개발자가 되고자 했던 이유

이전에는 패키지 제작관리 업무를 하며 생산 공정, 납기, 고객 대응을 담당했습니다. 당시 바코드 확인, 물품 입출고 확인, 다음 공정 진행 여부 확인처럼 반복적으로 확인해야 하는 업무가 많았고, 대부분 사람의 경험과 수작업에 의존하고 있었습니다.

반복 업무를 시스템으로 관리하면 누락을 줄이고 더 안정적으로 운영할 수 있겠다는 생각이 개발자로 전향한 계기가 되었습니다. 단순히 화면에 기능을 추가하는 데 그치지 않고, 실제 업무 흐름을 이해해 사람이 반복하던 일을 시스템으로 바꾸는 개발을 지향합니다.

관심 있는 개발 분야

외부 시스템 연동, 데이터 수집 자동화, 대량 데이터 처리, 운영 안정성을 높이는 백엔드 구조에 관심이 있습니다. ERP 와 외부 쇼핑몰·물류 시스템을 연동하며 외부 API 가 항상 정상 응답을 주지 않고, 지연·실패·중복 요청·Timeout 이 일상적으로 발생한다는 점을 경험했습니다.

이후 Easy Hub 에서 주문수집 요청을 Job 으로 관리하고 Kafka Worker 기반 비동기 처리, Outbox Pattern, Retry/DLQ, Recovery, DB Lock 을 적용했습니다. 앞으로도 사용자에게는 단순히 보이지만 내부적으로는 실패를 추적하고 안전하게 복구할 수 있는 시스템을 설계하는 개발자로 성장하고 싶습니다.

기억에 남는 개발 프로젝트

가장 기억에 남는 프로젝트는 개인 프로젝트 Easy Hub 입니다. 회사에서 REST API 기반 동기 주문수집 기능을 개발하며 외부 API 실패, 데이터 유실 가능성, 로그 신뢰성, 처리 완료 여부 추적의 어려움을 경험했고, 이를 직접 해결해 보기 위해 프로젝트를 시작했습니다.

주문수집 요청을 Job 으로 관리하고 Kafka 기반 비동기 파이프라인으로 확장했으며, Outbox Pattern 으로 DB 저장과 메시지 발행의 일관성을 높였습니다. Worker 장애 복구, Retry/DLQ, 동시성 제어, 모니터링까지 구현하고 Mock Mail 기반 100,000 건 E2E 부하 테스트에서 4 worker 처리량이 1 worker 대비 약 3.1 배 개선되는 것을 확인했습니다.

현재는 AWS EC2 에 운영 배포를 완료하고 Nginx 와 도메인을 연결해 실제 운영 주소에서 접속할 수 있도록 구성했습니다. 또한 수집 Job 별 Attempt 이력을 생성하고 종료 처리하는 구조를 구현해, 진행 중인 수집 과정과 각 시도의 완료·실패 여부를 관측할 수 있도록 개선했습니다.

이 프로젝트를 통해 안정적인 백엔드는 비동기 처리 구조뿐 아니라 운영 환경에서 상태를 관측하고 문제 원인을 추적할 수 있어야 한다는 점을 배웠습니다. 앞으로도 외부 시스템이 불안정한 상황에서 실패를 안전하게 복구하고, 사용자가 신뢰할 수 있는 결과를 제공하는 백엔드 개발자가 되겠습니다.